

HELPGRID

Detailed Product Requirements Document

Turn local problems into local solutions.

Prototype city: Indore, India Primary users: Requesters + Helpers Scope: Campus → City

HelpGrid is a location-aware coordination platform that connects people who have local problems, needs, resources or services with nearby people who can take action. It combines structured requests, AI-assisted understanding, intelligent matching, realtime coordination, proof and requester verification.

Area	Definition
Core request types	NEED • GIVE • SERVICE • REPORT
Core workflow	Create → Understand → Match → Accept → Act → Prove → Verify → Resolve
Frontend	React + React Router + Tailwind CSS
Backend direction	Node.js + Express + PostgreSQL + Prisma
Realtime	Socket.IO
Maps	Mapbox or Google Maps
Media	Cloudinary or S3-compatible storage
AI	Classification, duplicate detection, urgency, summaries and matching support

Purpose of this PRD

This document defines product behavior, user journeys, functional requirements, AI behavior, matching logic, screens, data model, architecture, security, MVP boundaries, testing and the final demonstration scenario. It is intended to be the single product reference for UI design and implementation.

1. Executive Summary

Local problems are often scattered across WhatsApp groups, phone calls, informal contacts and notice boards. The person who can solve a problem may never discover it, while people with useful skills or spare resources may not know where they are needed.

HelpGrid turns a local issue into an actionable, location-aware request. AI converts messy natural-language input into structured information; a matching engine finds suitable nearby helpers; realtime tools coordinate the task; proof and verification create accountability.

Product promise: the platform should move a valid request from problem to verified resolution with as few taps as possible.

1.1 Objectives

- Make creating a request fast enough for a first-time user to complete in under two minutes.
- Understand natural-language descriptions and propose category, urgency, affected people, deadline and keywords.
- Match requests with helpers using skill, distance, availability, experience and trust.
- Give helpers one obvious next action at every stage.
- Keep requester and helper coordinated through status, chat, ETA and arrival actions.
- Create evidence through before/after proof and requester verification.
- Build a scalable domain model that can start on campus and expand to city or multi-city use.

1.2 Success criteria

Metric	MVP signal
Creation	Request can be published in <2 minutes by a first-time user.
AI	Users mainly edit AI suggestions instead of manually filling long forms.
Matching	Recommended helpers have visible reasons for the match.
Completion	Accepted tasks can reach proof and verification without leaving the product.
Clarity	Every screen has one clear next action.
Consistency	The same request facts remain consistent across all routes.

2. Problem & Opportunity

2.1 Current problems

- Discovery: problems are visible only to a small informal network.
- Matching: there is no structured way to find the most relevant local person.
- Coordination: requester and helper repeatedly exchange location, ETA and status information.
- Accountability: completion is often based on a message such as “done” rather than evidence and verification.

2.2 Product opportunity

A single coordination layer can connect local demand and local capability. The product is intentionally not a generic social network, ticketing system or chatbot. The core unit is an actionable local request.

3. Vision, Principles & Scope

Vision: Build a trusted local coordination network where useful action can be discovered and completed close to where it is needed.

3.1 Principles

- Action over content: every important screen ends with a clear next action.
- AI as infrastructure: AI assists understanding and matching instead of dominating the UI.
- Local first: location, distance and availability are first-class data.
- Trust through evidence: task history, timestamps, proof and verification matter.
- Human control: AI suggestions are editable and never silently override user facts.
- Minimal friction: prefer natural language and progressive disclosure over long forms.
- Scalable foundation: prototype data can be seeded, but domain entities must support a real backend.

3.2 MVP scope boundary

The MVP focuses on two roles: Requester/User and Helper. Organizations and Admin are designed as future extensions, not core implementation requirements.

4. Users & Roles

Role	Goal	Core permissions	MVP
Requester / User	Get a local need/problem resolved	Create requests, track, chat, verify	Yes
Helper	Find and solve nearby problems	Availability, matches, accept, task, proof	Yes
Organization	Coordinate multiple helpers	Team management, assignment, analytics	Later
Admin	Maintain safety and quality	Moderation, users, disputes, platform analytics	Later

4.1 Request types

Type	Meaning	Example	Primary action
NEED	I need something	Need a book or transport help	Find resource/helper
GIVE	I have something useful	Spare charger or textbook	Match to a need
SERVICE	I can provide a service	Repair, tutoring, delivery	Publish capability
REPORT	Something is wrong	Lab PCs won't boot	Route to capable helper

5. Canonical Product Workflow

5.1 Main lifecycle

DRAFT → PUBLISHED → MATCHING → MATCHED → ACCEPTED → IN_PROGRESS → ARRIVED → PROOF_SUBMITTED → AWAITING_VERIFICATION → RESOLVED

Exception states: CANCELLED, EXPIRED, REWORK_REQUESTED and DISPUTED.

5.2 Canonical demo request

“Lab 3 PCs won't boot”
 Affected: 14 students
 Location: College Lab 3
 Urgency: High
 Type: REPORT

This exact request should be reused across Explore → Request Detail → Match → Task → Proof → Verification → Resolved. Additional derived data may be shown, but source facts must remain consistent.

5.3 End-to-end flow

Problem → AI Understanding → Match → Accept → Act → Proof → Verify → Resolve

6. User Journeys

6.1 Requester journey

Step	User action	System behavior
1	Choose type	Creates typed draft
2	Describe problem	AI extracts structured fields
3	Choose location	Stores coordinates + readable location
4	Review	User edits AI fields
5	Publish	Request becomes matchable
6	Matching	System calculates helper candidates
7	Helper accepts	Task becomes active
8	Coordinate	Chat, ETA, arrival/status
9	Proof	Helper uploads evidence
10	Verify	Confirm / rework / report
11	Resolve	Request closes with verified outcome

6.2 Helper journey

- Turn availability ON.
- See nearby opportunities ranked by fit and urgency.
- Open task and review exact requirement.
- Inspect match factors and accept the task.
- Use Start → Arrived → Complete actions.
- Submit before/after proof and a short note.
- Receive requester verification or a rework request.

7. Functional Requirements

ID	Area	Requirement
FR-01	Authentication	Signup, login, logout and profile.
FR-02	Role onboarding	Choose need help, want to help, want to give or organization.
FR-03	Skill onboarding	Free text skills + AI suggested tags.
FR-04	Request creation	Create NEED, GIVE, SERVICE or REPORT.
FR-05	Natural language	Primary request field accepts ordinary language.
FR-06	AI classification	Suggest category, urgency, keywords, affected count and summary.
FR-07	Duplicate detection	Warn about similar nearby active requests.
FR-08	Location	Map point + readable location.
FR-09	Review	AI UNDERSTANDING is editable before publishing.
FR-10	Explore	Browse and filter local requests.
FR-11	Matching	Generate explainable helper matches.
FR-12	Task lifecycle	Accept → Start → Arrived → Complete.
FR-13	Realtime	Chat and state updates.
FR-14	Proof	Before/after media + note.

ID	Area	Requirement
FR-15	Verification	Confirm resolution / rework / report problem.
FR-16	Notifications	In-app state-change notifications.
FR-17	Live map	Aggregated local activity.
FR-18	System states	Reusable loading, offline, error and no-match states.

8. AI Requirements

AI should be invisible when it is not useful and visible only when it improves a decision.

Capability	Input	Output	Control
Understanding	Natural-language request	Type/category/urgency/keywords/affected count	Edit
Duplicate detection	Current + nearby requests	Similarity warning	Ignore
Priority	Text + impact + deadline	Suggested urgency	Edit
Matching	Request + helper data	Score + explanation	Accept/ignore
Recommendations	Intent + local data	Relevant requests/helpers	Normal navigation
Summary	Request history	Short status summary	Read

8.1 Example

Input: "Lab 3 PCs won't boot and 14 students are waiting."

Suggested: Category = Computer Lab / Hardware; Urgency = High; Reason = Multiple students affected.

8.2 AI guardrails

- Never fabricate location, deadline, affected count or proof.
- Label AI-derived fields with a small AI tag.
- Keep original user text for audit/debugging.
- Allow edit or dismiss for every AI suggestion.
- Do not use AI score as the sole eligibility rule.
- Store model version for reproducibility in the academic project.
- Use deterministic fallback rules if the AI service is unavailable.

9. Matching Engine

9.1 Initial scoring model

Match Score = 35% Skill Fit + 25% Distance + 15% Availability + 15% Experience + 10% Trust

These are initial configurable assumptions, not validated scientific weights. Hard eligibility rules should run before scoring.

9.2 Match explanation

92% MATCH

Strong hardware skills • 0.8 km away • Available now • 18 completed tasks • Trusted profile

9.3 Eligibility

- Helper is available for the task.
- Required specialist skills are satisfied where applicable.
- Distance is within the request radius unless widened.
- Blocked/suspended/conflicting helpers are excluded.

- Urgent requests can require current availability.

10. Information Architecture & Routes

Route	Area	Purpose
/	Public	Landing: hero, request types, how it works, live request list, footer.
/explore	Public	Browse requests with type, distance and urgency filters.
/login	Public	Authentication.
/signup	Public	Account creation.
/onboarding/role	Onboarding	Primary role/intention.
/onboarding/skills	Onboarding	Skills + AI suggestions.
/app	User	Dashboard: active requests, recommendations, nearby.
/app/create	User	Pick NEED / GIVE / SERVICE / REPORT.
/app/create/details	User	Text input + AI parsing + duplicate warning.
/app/create/location	User	Map pin.
/app/create/review	User	AI UNDERSTANDING review.
/app/create/success	User	Published + matching animation.
/app/explore/map	User	Full-screen request map.
/app/request/:id	User	Request detail and primary action.
/app/matches/:id	User	Match score and factor breakdown.
/app/requests/:id	User	Tracking timeline.
/app/requests/:id/verify	User	Proof and verification.
/app/messages/:id	User	Task-aware chat.
/helper	Helper	Availability + nearby opportunities + active task.
/helper/tasks/:id	Helper	Task detail + next action.
/helper/tasks/:id/proof	Helper	Before/after proof.
/live	Signature	Live map, counters, near-you panel and AI feed.

Reusable system states

- No match: widen radius / notify me later.
- Offline: preserve draft and retry.
- Error: explain failure + one recovery action.
- Loading: shared skeleton pattern.
- Empty: short explanation + one relevant action.

11. Screen-Level Requirements

Screen	Requirements
Landing	Hero with Get Help / I Want to Help; 4 request types; 3-step flow; one live request list.
Explore	Dense request list, filters, distance and urgency; avoid oversized dashboard cards.
Role onboarding	Four clear intent choices.
Skills onboarding	Free-text skill input; AI suggestions as accept/dismiss chips.
Dashboard	Active requests, two AI recommendations, nearby activity.
Create type	Four clear tiles.

Screen	Requirements
Create details	One primary text box, live AI parsing, duplicate warning.
Location	Map pin + readable location + confirm.
Review	Editable AI UNDERSTANDING block.
Success	Reading → Searching nearby → Notifying helpers.
Request detail	Title, type, urgency, location, description, AI summary, timeline, helpers, action.
Matches	92% style score + skill/distance/availability/experience/trust.
Tracking	Created → Matching → Helper found → Accepted → In progress → Proof → Verified.
Verification	Before/after + Confirm / Rework / Report.
Messages	Task header + chat + ETA/location/arrived quick actions.
Helper home	Availability toggle + opportunities + active task.
Helper task	One next-step button: Accept → Start → Arrived → Complete.
Proof	Before/after upload + note.
Live	Dark full-screen map; counters; marker system; near-you panel; AI activity feed.

12. UI / UX Specification

Token	Specification
Background	#F5F2EC
Surface	#FFFFFF
Ink	#1A1916
Line	#E3DED4
Brand	#1F4D3A
Urgent	#C2412D
Pending	#B7791F
Resolved	#3E7B4F
Typography	Geist or Instrument Sans; serif only for landing H1; mono for IDs/times/distances
Shape	1px borders; 6px controls; 10px cards; avoid excessive rounding
Icons	Lucide, 1.5px stroke; no emoji icons
Desktop	1440px target; 232px sidebar + content
Mobile	390px; top bar + bottom nav: Home, Explore, Create, Messages, Profile

12.1 Avoid

- Gradients, glow, glassmorphism and decorative blobs.
- Centered hero over floating dashboard mockup.
- Three identical icon-circle feature cards.
- Huge rounded-2xl containers everywhere.
- Robot/sparkle AI imagery and generic stock team photos.
- Buzzword-heavy copy such as seamless, empower, unlock and leverage.
- Generic Bootstrap/SaaS dashboard aesthetic.

12.2 Interaction rules

- One main action per screen.
- Titles preferably under 8 words; descriptions under 20.
- Use action verbs such as Accept task and Confirm resolution.
- Use relative times such as 12 min ago.
- Not every request needs an image.
- Motion target ≤240ms; respect prefers-reduced-motion.

12.3 Motion

Page fade-up; match score ring 0→value; timeline current step pulse; AI review fields fill sequentially; success sequence; live-map marker ripples and counter ticks. No parallax, bounce, confetti or carousels.

13. Shared Component System

Component	Responsibility
Button	Primary, secondary, destructive and quiet actions.
Input	Text, search, select and validation states.
Tabs	Filter/state navigation.
RequestCard	Shared request summary.

Component	Responsibility
StatusChip	Consistent status language and color.
AIBlock	AI-derived information with AI tag and editable fields.
Timeline	Request/task lifecycle.
MatchScore	Score + factor breakdown.
Map	Reusable map and marker system.
Toast	Short feedback.
Skeleton	Single loading language.
BottomSheet	Mobile contextual detail/actions.
Navigation	Desktop sidebar + mobile navigation.

14. Data Model

Entity	Core fields
User	id, name, email, phone, avatar, role, city, trustScore, createdAt
Skill	id, name, category
UserSkill	userId, skillId, level, verified
Request	id, creatorId, type, title, description, category, urgency, status, lat, lng, locationText, deadline, affectedCount, createdAt
RequestAI	requestId, categorySuggestion, urgencySuggestion, summary, duplicateScore, modelVersion
Match	id, requestId, helperId, score, skillFit, distance, availability, experience, trust, explanation
Task	id, requestId, helperId, status, acceptedAt, startedAt, arrivedAt, completedAt
Message	id, taskId, senderId, text, createdAt, readAt
Proof	id, taskId, type, mediaUrl, note, uploadedAt
Verification	id, requestId, requesterId, outcome, comment, createdAt
Notification	id, userId, type, title, body, readAt, createdAt
Availability	userId, isAvailable, radiusKm, updatedAt

Relationships

User 1→N Request; Request 1→N Match; User 1→N Match; Request 1→0/1 Task; Task 1→N Message; Task 1→N Proof; Request 1→N Verification; User N→N Skill.

15. Technical Architecture

Layer	Technology	Responsibility
Frontend	React + React Router + Tailwind	UI, routing, forms, map and realtime client.
Backend	Node.js + Express	REST API, authorization, business rules.
Database	PostgreSQL + Prisma	Persistent relational state.
Realtime	Socket.IO	Chat, status and availability updates.
Maps	Mapbox or Google Maps	Tiles, geocoding, markers, distance.
Media	Cloudinary / S3	Proof images and optional media.
AI	LLM/API + fallbacks	Understanding, duplicate detection, summaries, recommendations.

Layer	Technology	Responsibility
Notifications	In-app first; Email/WhatsApp/SMS optional	State-change notifications.

15.1 API direction

Auth: POST /auth/signup, POST /auth/login, POST /auth/logout
Requests: GET /requests, POST /requests, GET/PATCH /requests/:id
AI: POST /ai/understand-request, POST /ai/check-duplicate
Matches: GET /requests/:id/matches, POST /matches/:id/accept
Tasks: GET /tasks/:id, PATCH /tasks/:id/status, POST /tasks/:id/proof
Messages: GET/POST /tasks/:id/messages
Verification: POST /requests/:id/verify
Availability: PATCH /helpers/me/availability

15.2 State authority

Critical task transitions must be validated server-side. The browser must never be trusted to mark a task resolved without required proof/verification.

16. Security, Trust & Privacy

- Hash passwords securely; never store plaintext credentials.
- Use secure authentication/session or JWT handling.
- Enforce role-based authorization and ownership checks.
- Validate uploads by type and size; use controlled storage access.
- Do not expose unnecessary exact home-location data publicly.
- Rate-limit authentication, messaging and request creation.
- Sanitize user-generated content.
- Audit important state transitions with timestamps.
- AI output is not verified fact until confirmed by a user/system rule.

17. Non-Functional Requirements

Area	Requirement
Performance	Fast API responses under normal prototype load; immediate UI skeletons.
Availability	Core request flow degrades gracefully if AI/realtime fails.
Scalability	Data model supports multiple cities.
Accessibility	Keyboard access, readable contrast, labels, focus and reduced motion.
Responsive	1440px desktop and 390px mobile are primary targets.
Observability	Log API errors, AI failures, task transitions and backend exceptions.
Maintainability	Shared components, centralized data, clear route/domain boundaries.
Consistency	Canonical request record is the source of truth.

18. MVP Scope

Must have

- Authentication and onboarding.
- Four request types.
- Natural-language request creation.
- AI understanding with editable output.
- Location selection and request explore.
- Helper availability and transparent matching.
- Task state machine and messaging.
- Proof upload and requester verification.
- Responsive UI and reusable system states.
- Seeded canonical demo data.

Should have

- Duplicate detection
- Push/email notifications
- Basic trust score
- Live map
- Basic response/completion analytics

Out of MVP

- Full organization management

- Complex admin console
- Payments
- Government integrations
- Large-scale ML training
- Multi-language AI
- Advanced dispute system
- Public social feed

19. Build Plan

Phase	Focus	Deliverables
1	Foundation	Tokens, routing, components, mock data, auth.
2	Request creation	Type, details, AI, location, review, success.
3	Discovery	Explore, request detail, filters, map.
4	Matching	Helper profile, availability, score and explanation.
5	Task execution	States, chat, ETA, arrived/start/complete.
6	Proof & verification	Media, verification, rework and resolved state.
7	Signature	HelpGrid Live map, counters and activity feed.
8	Hardening	Validation, auth rules, errors, mobile, testing, deployment.

20. Final Demo Scenario

The strongest B.Tech demonstration is one complete request journey rather than disconnected CRUD pages.

Scenario

A student reports: “Lab 3 PCs won't boot.” Fourteen students are blocked. The request is categorized as REPORT, marked High urgency and pinned to College Lab 3.

Moment	Screen	Demonstration
1	Create	Natural language starts the request.
2	AI Review	Category, urgency and impact are proposed and editable.
3	Publish	Matching animation communicates routing.
4	Match	Nearby hardware-skilled helper gets explainable 92% match.
5	Accept	Task state updates.
6	Coordinate	ETA, arrival and chat.
7	Proof	Before/after evidence uploaded.
8	Verify	Requester confirms or requests rework.
9	Resolved	Timeline ends in verified resolution.
10	Live	Same activity appears on HelpGrid Live.

Academic value

- Full-stack domain workflow beyond basic CRUD.
- Meaningful AI integration.
- Geolocation and matching logic.
- Realtime communication.
- Media upload and verification.

- Measurable technical components for a major-project report.

21. Testing Strategy

Test type	Scope
Unit	Matching score, validation, state transitions, AI normalization.
API	Auth, authorization, request CRUD, matching, tasks, verification.
Integration	Create → match → accept → proof → verify.
UI	Forms, loading/errors, navigation, responsive behavior.
Realtime	Message and task-state delivery.
Security	Unauthorized edits, upload validation, rate limits, ownership.
Usability	First-time requester can publish without explanation; helper understands next action.
Demo data	Canonical request remains consistent across screens.

22. Risks & Mitigations

Risk	Potential issue	Mitigation
AI hallucination	Wrong structured data	Editable suggestions + schema validation + fallbacks.
Poor matching	Helper cannot act	Hard eligibility rules + transparent score.
Location privacy	Overexposure	Task-appropriate location precision.
Realtime failure	Updates delayed	Database remains source of truth.
Fake proof	Evidence insufficient	Requester verification + future moderation.
Scope creep	Too many features	Keep requester/helper loop as MVP.
Generic UI	Looks like dashboard template	Editorial layouts, maps, dense useful lists, task-specific actions.

23. Future Roadmap

Stage	Expansion
Campus	Students, labs, books, resources and campus helpers.
City	Neighborhoods, larger helper network and local organizations.
Organizations	Team accounts, assignment queues, service targets, analytics.
Intelligence	Learn from completed matches to improve recommendations.
Trust	Verified skills, identity levels, history and dispute resolution.
Infrastructure	Multi-city tenancy, stronger observability and scalable realtime.
Community	Optional reputation/community features after the coordination loop is proven.

24. Product Definition of Done

HelpGrid is successful when a user can describe a real local problem in ordinary language, the system understands what is needed, a suitable nearby helper discovers it, both parties coordinate, the helper provides evidence, and the requester verifies the outcome — all inside one coherent workflow.

The product is defined by this loop:

PROBLEM → UNDERSTAND → MATCH → ACT → PROVE → VERIFY → RESOLVE

Every other feature — dashboard, AI summary, map, chat, notifications and analytics — exists to make this loop faster, clearer, safer and more trustworthy.

Appendix A — Prototype Seed Data

Object	Seed value
City	Indore, India
Canonical request	Lab 3 PCs won't boot
Affected	14 students
Location	College Lab 3
Urgency	High
Type	REPORT
Nearby helper	Hardware-skilled, available now, ~0.8 km
Example match	92% with factor breakdown
Live counters	127 Active Requests • 43 Helpers Online • 18 Organizations Online

Appendix B — Product language

Prefer	Avoid
Accept task	Submit
Confirm resolution	Close ticket
Request rework	Reopen case
Helper	Agent/operator
Request	Ticket only
AI understanding	AI magic
12 min ago	12 minutes ago / timestamp overload